Data Engineering

Managing Multi-Sensor Data at Scale

Phase 11 — Research-Grade Notes

Trinetra-AI

An Intelligent Multi-Sensor Fusion Framework for GPS-Denied Navigation

Data engineering is the unglamorous foundation that everything else stands on. A navigation system generates *torrents* of multi-rate sensor data — **accelerometer, gyroscope, magnetometer, GPS, barometer** at 1–1000 Hz — that must be *captured, stored, synchronised, cleaned, versioned, and served* to the AI modules and fusion core reliably and reproducibly. This document builds that data infrastructure: formats, databases, pipelines, schemas, quality, feature stores, and versioning — always for the concrete workload of GPS-denied multi-sensor navigation.

Capture → store → model → clean → serve → version — reliably and reproducibly.

Contents

I Foundations **2**

1 Introduction to Data Engineering **2**

1.1 What is data engineering? . . . 2

1.2 Why navigation & robotics generate "big" data . . . 2

1.3 The sensor data lifecycle . . . 2

1.4 Streaming vs batch, edge vs cloud . . . 2

II Storage **3**

2 Sensor Data Formats **3**

2.1 The format landscape . . . 3

2.2 Row vs columnar, and why it matters . . . 3

2.3 ROS Bag, rosbag2, and MCAP . . . 4

3 Databases **4**

3.1 The database landscape . . . 5

3.2 Relational vs time-series vs NoSQL . . . 5

III Movement & Modeling **5**

4 Data Pipelines **6**

4.1 ETL vs ELT, batch vs streaming . . . 6

4.2 Tools . . . 6

5 Data Modeling **7**

5.1 Schema, normalization, partitioning . . . 7

5.2 A time-series schema for sensor data . . . 7

6 Time-Series Data Engineering **7**

IV Quality, Features & Governance **8**

7 Data Quality **8**

8 Feature Store **9**

9 Data Versioning **10**

10 Data Security **10**

V Resources & Practice **11**

11 Research Datasets **11**

12 Practical Implementation **11**

12.1 The complete Trinetra-AI data architecture 12

12.2 Repository / folder structure 12

12.3 Implementation notebooks 12

13 Interview & Research Preparation **13**

13.1 50 interview questions 13

13.2 30 viva questions 14

13.3 20 coding questions 15

14 Summary, Glossary & Roadmap **16**

14.1 Summary 16

14.2 Glossary 16

14.3 Roadmap: Data Engineering → Visualization 16

Part I — Foundations

1 Introduction to Data Engineering

1.1 What is data engineering?

Data engineering is the discipline of designing and building the systems that *collect, store, transform, and serve* data reliably and at scale. Where data *science* asks "what can we learn?", data engineering ensures the data is *available, correct, timely, and reproducible* for that learning. In a robotics/navigation context it is the plumbing between raw sensors and the AI/fusion algorithms.

Intuition: the AI modules (Phase 9) and fusion core (Phase 7) are the engine; data engineering is the fuel system — tanks (storage), pipes (pipelines), filters (quality), and gauges (metadata). A brilliant model starved of clean, well-organised data will fail just as surely as an engine with a clogged fuel line.

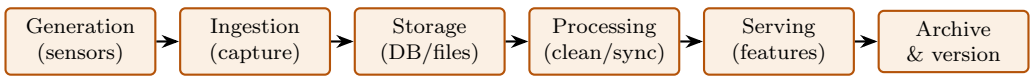
1.2 Why navigation & robotics generate "big" data

A 9-DOF IMU at 200 Hz, stored as `float64` (8 bytes) per channel plus a timestamp, produces

$$200\,\text{Hz} \times (9 \times 8 + 8)\,\text{B} = 200 \times 80 = 16{,}000\ \text{B/s} \approx 57.6\ \text{MB/hour}.$$

Add GPS, barometer, magnetometer, and (future) camera/LiDAR, and a single logging session is hundreds of MB to many GB. Across dozens of trajectories for training, the corpus reaches tens–hundreds of GB. Two consequences: (1) **format matters** — Parquet+Zstd compresses this 5–10×; (2) **throughput matters** — at 200 Hz the ingest pipeline has a 5 ms budget per sample, or messages drop.

1.3 The sensor data lifecycle



1.4 Streaming vs batch, edge vs cloud

Axis	Meaning	Trinetra-AI choice
Batch	process stored data in bulk	training, offline analysis
Streaming	process data as it arrives	live navigation, health monitoring
Real-time	bounded-latency streaming	the on-vehicle fusion loop
Edge	compute on the device (Jetson)	live inference, low latency
Cloud/offline	compute on a server	training, storage, versioning

Trinetra-AI has *two* data regimes: an **offline/batch** pipeline (collect logs, store as Parquet/MCAP, clean, version, train the AI modules) and an **online/streaming** pipeline (on the Jetson: capture at rate, sync, run inference, fuse — in real time). Both are built here. The offline pipeline feeds Phase 9's models; the online pipeline runs Phase 7–8's fusion/navigation live.

Part II — Storage

2 Sensor Data Formats

The format decision governs storage size, read/write speed, schema safety, and tool compatibility. There is no single winner — Trinetra-AI uses different formats at different stages.

2.1 The format landscape

Format	Type	Schema	Compress	Columnar	Best for
CSV	text	no	poor	no	tiny data, human-readable, interchange
JSON	text	loose	poor	no	config, metadata, nested records
Parquet	binary	**yes**	**great**	**yes**	**analytics / ML tables**
ORC	binary	yes	great	yes	Hadoop/Hive analytics
Avro	binary	yes	good	no (row)	streaming, schema evolution
Protobuf	binary	yes	good	no	wire messages, ROS/-gRPC
HDF5	binary	yes	good	n-D arrays	large scientific arrays
ROS Bag (.db3)	binary (SQLite)	msg	optional	no	legacy ROS 2 logs
MCAP (.mcap)	binary	msg	**LZ4/Zstd**	indexed	**ROS 2 sensor logs (default)**

2.2 Row vs columnar, and why it matters

Row-oriented formats (CSV, Avro) store record-by-record — efficient for writing whole records and streaming. **Columnar** formats (Parquet, ORC) store column-by-column — efficient for reading *some* columns of *many* rows and for compression (adjacent values are similar). ML/analytics workloads ("give me the gyro-z of all rows in this window") are columnar-friendly, which is why **Parquet** dominates the ML data layer.

2.3 ROS Bag, rosbag2, and MCAP

A **ROS bag** is the standard container for recorded ROS message streams (topics + timestamps). ROS 2's **rosbag2** is plugin-based; two storage backends matter: **SQLite3** (`.db3`, the historical default) and **MCAP** (`.mcap`). Since ROS 2 *Iron* (2023), **MCAP is the default and recommended format** — a chunked, indexed, append-only container with per-chunk LZ4/Zstd compression, embedded schemas, fast time-based seeking, and serialization-agnostic storage (ROS 1/2, Protobuf, JSON). SQLite3 remains supported for compatibility.

Trinetra-AI's format strategy: record live sensor streams as **MCAP** (native ROS 2, indexed, compressed, replayable in Foxglove); convert to **Parquet** for the ML/training data layer (columnar, fast per-channel reads, pandas/PyArrow-native); keep **JSON** for run metadata/config and **HDF5** only if storing large raw camera/LiDAR arrays later. CSV is used only for tiny human-inspectable exports. This "MCAP for capture, Parquet for compute" split is the practical backbone.

```
import pyarrow as pa, pyarrow.parquet as pq, pandas as pd
# Convert a synced sensor DataFrame to compressed columnar Parquet
df.to_parquet("session_042.parquet", engine="pyarrow", compression="zstd")
# Read back only the columns you need (columnar advantage)
gyro = pq.read_table("session_042.parquet", columns=["t","gyro_x","gyro_y","gyro_z"])
```

```
# Record ROS 2 sensor topics to MCAP (default since Iron); Zstd chunk compression
ros2 bag record -o session_042 --storage mcap \
    --compression-mode file --compression-format zstd /imu /mag /gps /baro
ros2 bag info session_042 # inspect topics, rates, duration
```

3 Databases

Files are for bulk storage; *databases* add query, indexing, concurrency, and — for sensor data — time-series superpowers.

3.1 The database landscape

Database	Type	Strength	Trinetra-AI use
SQLite	embedded SQL	zero-config, single file	on-device local store, dev
PostgreSQL	relational SQL	robust, extensible, JSON	central metadata & results
TimescaleDB	Postgres + TS	**hypertables, time partitioning**	**sensor time-series store**
InfluxDB	TS (custom)	fast ingest, retention policies	live metrics / monitoring
DuckDB	embedded OLAP	in-process analytics on Parquet	fast local analysis
MongoDB	document (NoSQL)	flexible nested docs	config / heterogeneous logs
Redis	in-memory KV	sub-ms cache, streams	online feature cache / buffer
Vector DB (FAISS, Milvus)	similarity	nearest-neighbour on embeddings	map/feature retrieval (future)

3.2 Relational vs time-series vs NoSQL

A **relational** database stores typed rows in tables linked by keys, queried with SQL (strong consistency, joins). A **time-series** database is optimised for timestamp-indexed, append-heavy data (automatic time partitioning, downsampling, retention). A **NoSQL** store trades schema/joins for flexibility or speed (document, key-value, column-family, graph). Sensor readings are overwhelmingly time-series; metadata and results are relational.

```
-- TimescaleDB: a hypertable auto-partitions sensor readings by time
CREATE TABLE imu_readings (
  t TIMESTAMPTZ NOT NULL,
  session_id INT NOT NULL,
  ax REAL, ay REAL, az REAL, gx REAL, gy REAL, gz REAL
);
SELECT create_hypertable('imu_readings', 't'); -- time partitioning
CREATE INDEX ON imu_readings (session_id, t DESC); -- fast per-session queries
-- Continuous aggregate: 10 Hz downsample for dashboards
SELECT time_bucket('100ms', t) AS bucket, avg(gz) FROM imu_readings GROUP BY bucket;
```

Trinetra-AI's storage tiers: TimescaleDB (Postgres extension) as the sensor time-series store — hypertables give automatic time partitioning, fast windowed queries, and continuous aggregates for dashboards; plain **PostgreSQL** tables for sessions, calibration params, model runs, and results; **SQLite** on-device for a lightweight local buffer; **Redis** as the online feature cache in the live loop. This keeps one SQL dialect (Postgres) across the offline stack while adding time-series speed where it counts.

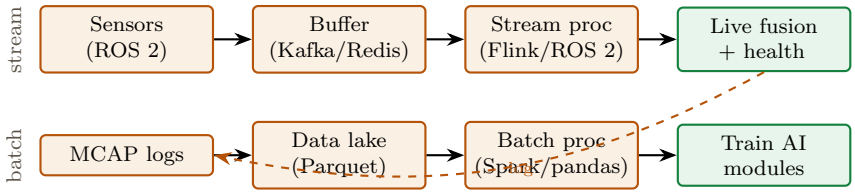
Part III — Movement & Modeling

4 Data Pipelines

A *pipeline* moves data from source to destination while transforming it. Trinetra-AI needs both a batch pipeline (training) and a streaming pipeline (live navigation).

4.1 ETL vs ELT, batch vs streaming

ETL (Extract–Transform–Load) transforms data *before* loading it into the store; **ELT** loads raw first, then transforms in-place (common with cheap columnar storage). **Batch** pipelines process bounded chunks on a schedule; **streaming** pipelines process unbounded data continuously with bounded latency; **real-time** adds a hard latency budget.



4.2 Tools

Tool	Role	Trinetra-AI fit
Apache Kafka	durable streaming message bus	buffer live sensor streams (scale)
Apache Spark	distributed batch/ETL	large-corpus batch processing
Apache Flink	low-latency stream processing	real-time feature/health streams
Apache Airflow	workflow orchestration (DAGs)	schedule training/ETL jobs
ROS 2 (rclpy)	real-time robotics middleware	the on-vehicle streaming pipeline
pandas / Polars	in-memory dataframes	per-session batch transforms

Right-sized for a research project: you almost certainly do *not* need Kafka/Spark/Flink for a single-vehicle thesis — that is production-scale machinery. Trinetra-AI's realistic pipeline is: **ROS 2** (rclpy) for live streaming capture → **MCAP** logs → **pandas/Polars** batch transforms → **Parquet** + **TimescaleDB**, orchestrated by simple scripts (or Airflow if runs multiply). Mention Kafka/Spark/Flink as the *scale-out* path, but keep the actual build lean.

```python
# Minimal ROS 2 subscriber that streams IMU into a buffer (rclpy sketch)
import rclpy; from rclpy.node import Node; from sensor_msgs.msg import Imu
class ImuIngest(Node):
    def __init__(self, sink):
        super().__init__("imu_ingest"); self.sink = sink
        self.create_subscription(Imu, "/imu", self.cb, qos_profile=50)
    def cb(self, m):
        t = m.header.stamp.sec + m.header.stamp.nanosec*1e-9
        self.sink.append((t, m.linear_acceleration.x, m.linear_acceleration.y,
                          m.linear_acceleration.z, m.angular_velocity.x,
                          m.angular_velocity.y, m.angular_velocity.z))
```

5 Data Modeling

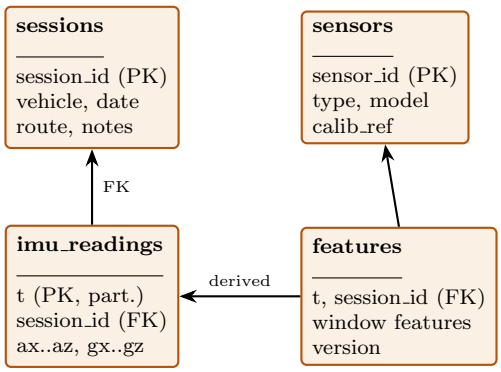
How you *structure* the data determines query speed, storage cost, and how easily the AI modules can consume it.

5.1 Schema, normalization, partitioning

A **schema** defines tables, columns, types, and relationships. **Normalization** removes redundancy by splitting into related tables (good for writes / integrity); **denormalization** duplicates data into wide tables (good for read-heavy analytics). **Indexes** speed lookups on chosen columns; **partitioning** splits a table by a key (time, session) so queries scan less; **compression** shrinks storage (and often speeds reads) at CPU cost.

5.2 A time-series schema for sensor data

Sensor *readings* are queried in huge windowed scans, so keep them in a **wide, partitioned, columnar** table (denormalized: one row = one timestamp with all fast-channel values), partitioned by time and `session_id`. Sensor *metadata* (device, calibration, mounting) changes rarely and is **normalized** into small reference tables joined by key. This hybrid — normalized cold metadata, denormalized hot readings — minimises both storage redundancy and query cost.



Trinetra-AI's model: small normalized **sessions** and **sensors** reference tables; a large partitioned **readings** hypertable (TimescaleDB) for the hot path; and a **features** table (versioned) holding the windowed features the AI modules consume. Partition readings by time + session so a "give me session 42's gyro over 10–20 s" query scans one small chunk, not the whole corpus.

6 Time-Series Data Engineering

Sensor data is time-series first, so the engineering revolves around *time*: stamping, aligning, resampling, and repairing it. (This operationalises Phase 4's concepts as a data pipeline.)

Operation	What it does	Trinetra-AI concern
Timestamp management	assign a consistent clock	hardware vs receipt time; monotonicity
Synchronization	align multi-rate streams	IMU 200 Hz vs GPS 1 Hz (Phase 4)
Resampling	change sample rate	up/down-sample to a common grid
Interpolation	fill between samples	linear/spline for slow channels
Missing-data handling	gaps, dropouts	mask (don't silently fill long GPS gaps)
Outlier handling	spikes, multipath	flag/gate (feed anomaly module)
Windowing	segment into fixed windows	training tensors (Phase 4)

Given streams on different clocks, choose a reference grid (usually the fastest sensor's) and interpolate each channel onto it: $x^{\text{grid}} = \text{interp}(t_{\text{grid}}, t_x, x)$. *Never* interpolate across a long dropout — mask it instead, so the fusion filter enters GPS-denied mode rather than fusing fabricated data. Sub-millisecond alignment errors at high dynamics appear downstream as "sensor error" that is really a *timing* error — the most common silent bug in a fusion stack.

```python
import numpy as np, pandas as pd
def align_to_grid(streams, rate_hz):
    """streams: {name: DataFrame(t, ...cols)} -> one aligned DataFrame."""
    t0 = max(s['t'].iloc[0] for s in streams.values())
    t1 = min(s['t'].iloc[-1] for s in streams.values())
    grid = np.arange(t0, t1, 1.0/rate_hz); out = {'t': grid}
    for name, s in streams.items():
        for c in s.columns.drop('t'):
            out[f"{name}_{c}"] = np.interp(grid, s['t'], s[c])
    return pd.DataFrame(out) # gaps handled separately by masking
```

Part IV — Quality, Features & Governance

7 Data Quality

Garbage in, garbage out — quality control is what protects the AI modules and fusion core from bad data.

The dimensions of **data quality**: **validity** (values in allowed ranges/types), **completeness** (no missing required fields), **consistency** (agrees across sources/units), **accuracy** (matches reality), **uniqueness** (no unintended duplicates), and **timeliness** (fresh, correctly ordered). Quality is enforced by *validation rules* at ingestion and *monitoring* over time.

Check	Rule	Trinetra-AI example
Range / validity	value within physical bounds	$\lvert a\rvert < 16g$, $\lvert\omega\rvert < 2000^\circ/\text{s}$
Rate / timeliness	samples arrive on time	IMU $\approx$ 200 Hz, no long gaps
Monotonic time	timestamps non-decreasing	catch clock resets
Completeness	required channels present	all 9 IMU axes non-null
Duplicate detection	no repeated (t, sensor)	de-dup on primary key
Cross-consistency	units & frames agree	accel in m/s^2, mag in μT
Statistical drift	distribution stable	flag data drift (Phase 9)

```python
import numpy as np
def validate_imu(df, fs=200):
    issues = []
    if (np.abs(df[["ax","ay","az"]]) > 16*9.81).any().any(): issues.append("accel out of range"
        )
    if (df["t"].diff().dropna() <= 0).any(): issues.append("non-monotonic time")
    dt = df["t"].diff().median()
    if abs(1/dt - fs) > 0.1*fs: issues.append("rate off nominal")
    if df[["ax","ay","az","gx","gy","gz"]].isna().any().any(): issues.append("missing channels"
        )
    return issues # empty list == clean
```

Data-quality checks run at *ingestion* (reject/flag bad sessions) and feed the *online* health/anomaly modules (Phase 9) at run time — the same validity rules that gate a training session also gate a live measurement before fusion. Tools like Great Expectations or Pandera can formalise these as versioned "data contracts" for reproducibility.

8 Feature Store

A **feature store** is a system that computes, stores, versions, and serves the *features* that models consume — with a crucial guarantee: the **offline** features used for training and the **online** features served at inference are *identical* (no train/serve skew). It also enables feature reuse, versioning, and point-in-time-correct joins (no future leakage).

Concept	Meaning	Trinetra-AI example
Offline store	historical features for training	windowed IMU features in Parquet
Online store	low-latency features for inference	latest window in Redis (edge)
Feature versioning	track feature definitions	"rolling-std v2" for drift model
Point-in-time join	no future leakage	join label at its own timestamp
Materialization	precompute & cache	nightly feature build

Do you need a full feature store (e.g. Feast)? For a single-vehicle thesis, a *lightweight* version suffices: a versioned **features** table (offline, Parquet / TimescaleDB) plus a **Redis** cache (online) that computes the *same* windowed features (rolling stats, FFT bands, lags from Phase 4). The non-negotiable principle is **train/serve parity**: the exact function that builds training features must build live features, or the drift/confidence models silently degrade in deployment. Reach for Feast only if the feature set and team grow.

```python
# One feature function, used BOTH offline (training) and online (inference) -> no skew
def window_features(w): # w: (L, C) window, shared code path
    import numpy as np
    fft = np.abs(np.fft.rfft(w, axis=0))
    return {"mean": w.mean(0), "std": w.std(0),
            "rms": np.sqrt((w**2).mean(0)),
            "dom_freq": fft.argmax(0),
            "band_low": fft[:len(fft)//4].sum(0)}
```

9 Data Versioning

Reproducibility is the currency of research: a result you cannot reproduce is not a result. Versioning data, code, and experiments is what makes Trinetra-AI's claims defensible.

Data versioning tracks changes to datasets so any result can be traced to the exact data that produced it. **Git** versions code (not large data); **DVC** (Data Version Control) versions large data/models by storing *pointers* in Git and blobs in remote storage; **LakeFS** gives git-like branching over a data lake; **MLflow** tracks experiments (params, metrics, artifacts) and registers models.

Tool	Versions	Trinetra-AI use
Git	code, configs, small files	the whole codebase
DVC	datasets, models (Git-linked)	versioned sensor corpus & checkpoints
LakeFS	data-lake branches	optional, larger-scale data ops
MLflow	experiments, models	track training runs, metrics, ablations
Config (Hydra)	experiment configs	reproducible run settings

Minimum reproducibility stack for the thesis: Git for code, **DVC** for the sensor dataset and model checkpoints (so "model X was trained on dataset v3" is provable), and **MLflow** to log every training run's params/metrics/artifacts. This is what lets you (and reviewers) reproduce an ablation months later and trust the numbers — directly addressing the citation/rigor concerns that matter for a research-grade project.

```
dvc init && dvc add data/sessions/ # version the sensor corpus (pointer in Git)
git add data/sessions.dvc .gitignore && git commit -m "dataset v3"
dvc remote add -d store s3://trinetra-data # blobs live in remote storage
# training run logged to MLflow (params, metrics, model artifact) for reproducibility
```

10 Data Security

Navigation data is sensitive — trajectories reveal location and behaviour, and the application domain (defense, autonomous systems) raises the stakes.

Core controls: **encryption at rest** (stored data) and **in transit** (TLS); **access control** (authentication + role-based authorization, least privilege); **backup & recovery** (regular backups, tested restores, disaster recovery); **privacy** (anonymise/aggregate location traces); and **integrity** (checksums, audit logs, immutable archives).

Control	Method	Trinetra-AI relevance
Encryption at rest	AES-256 on disk/DB	protect stored trajectories
Encryption in transit	TLS on all links	secure sensor→server streams
Access control	RBAC, least privilege	who can read raw traces
Backup / recovery	3-2-1 rule, tested restores	no lost training corpus
Integrity	checksums, audit log, immutable archive	tamper-evidence
Privacy	anonymise / aggregate	remove identifying locations

For a research project the security bar is proportionate: encrypt the corpus at rest, use TLS for any network transfer, keep versioned backups (DVC remote + a copy), and checksum archives for integrity. The *defense-relevant* framing matters for positioning the project — but do not over-engineer classified-grade controls for a thesis dataset. State the considerations; implement the sensible minimum.

Part V — Resources & Practice

11 Research Datasets

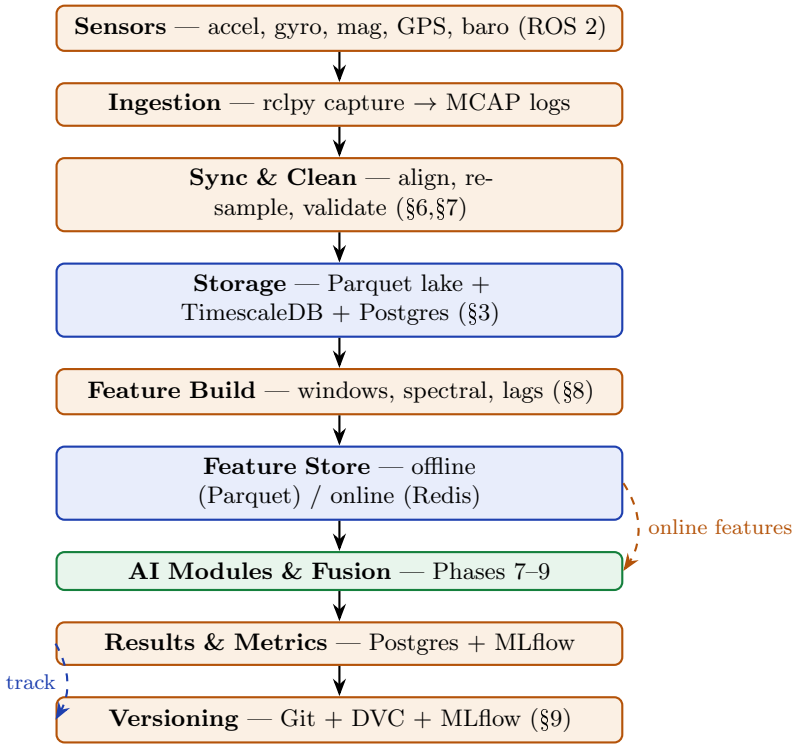
From the data-engineering angle, what matters is each set's *format*, *size*, and *how you ingest it*. All are real/verified; confirm licence on the official page.

Dataset	Native format	Scale	Ingestion note
KITTI	text + binary + PNG	∼tens of GB	parse OXTS/IMU-GPS text; per-drive
EuRoC MAV	ROS bag / CSV	GB-class	rosbag2 or CSV → Parquet
TUM VI	ROS bag / raw	large	bag → align → Parquet
RoNIN	HDF5 / CSV	GB-class	HDF5 reader → windows
TLIO	numpy / HDF5	GB-class	array store → tensors
OxIOD	CSV	moderate	direct CSV → Parquet
comma2k19	numpy chunks	∼100 GB	chunked loader; sample first
MagPIE	CSV / MAT	moderate	parse → Parquet; mag focus
DAF-MIT AIA MagNav	HDF5 / CSV	moderate	flight lines → Parquet

Data-engineering takeaway: regardless of native format, funnel everything into one **canonical schema** (synced, unit-normalised, Parquet+TimescaleDB) with a small **loader per dataset**. This "adapters → canonical store" pattern means the AI modules and fusion core see one consistent interface no matter which dataset trained them — essential for cross-dataset generalisation studies.

12 Practical Implementation

12.1 The complete Trinetra-AI data architecture



12.2 Repository / folder structure

```
trinetra-ai/
|- data/ # DVC-tracked (pointers in git, blobs in remote)
| |- raw/ # original datasets + MCAP session logs
| |- interim/ # synced/cleaned Parquet
| +- features/ # versioned feature tables
|- src/
| |- ingest/ # rclpy capture, dataset adapters
| |- pipeline/ # sync, clean, validate, windowing
| |- store/ # DB schema, Parquet/TimescaleDB IO
| |- features/ # ONE feature module (train == serve)
| +- models/ # Phase 9 AI modules
|- db/schema.sql # Postgres/TimescaleDB DDL
|- conf/ # Hydra configs (reproducible runs)
|- notebooks/ # analysis (numbered)
|- dvc.yaml .dvc/ # data pipeline stages & versions
+- mlruns/ # MLflow experiment tracking
```

12.3 Implementation notebooks

Notebook	Contents	Output
01_ingest_mcap.ipynb	rclpy/rosbag2 capture → MCAP	session logs
02_adapters.ipynb	per-dataset loaders → canonical	unified raw tables
03_sync_clean.ipynb	align, resample, validate	clean Parquet
04_schema_load.ipynb	DDL + load TimescaleDB/Postgres	queryable store
05_features.ipynb	windowed feature build (shared fn)	versioned features
06_quality_report.ipynb	data-contract checks	quality dashboard
07_versioning.ipynb	DVC + MLflow wiring	reproducible pipeline

Evaluation / best practices. Measure ingest throughput (msgs/s vs sensor rate; no drops), storage footprint (raw vs Parquet+Zstd), query latency (windowed scans), and pipeline reproducibility (re-run → identical features). **Best practices:** one canonical schema; train/serve feature parity; partition by time+session; version data & runs; mask (never fabricate) missing data; keep a raw immutable copy. **Common mistakes:** silently interpolating GPS gaps; train/serve feature skew; unversioned datasets (irreproducible results); wrong units/frames on ingest; timestamp/clock confusion.

13 Interview & Research Preparation

13.1 50 interview questions

1. Define data engineering vs data science.
2. Why do navigation systems generate big data?
3. Estimate a 9-DOF IMU's hourly data rate.
4. Sensor data lifecycle stages.
5. Batch vs streaming vs real-time.
6. Edge vs cloud processing.
7. Row vs columnar storage.
8. Why is Parquet good for ML?
9. CSV vs Parquet trade-offs.
10. What is Avro used for?
11. Protobuf vs JSON.
12. What is HDF5 good at?
13. What is a ROS bag?
14. rosbag2 storage backends.
15. Why MCAP over SQLite3 for logs?
16. What is a hypertable?
17. TimescaleDB vs InfluxDB.
18. SQLite vs PostgreSQL.
19. When to use Redis.
20. What is DuckDB good for.
21. When a vector DB?
22. Relational vs time-series vs NoSQL.
23. ETL vs ELT.
24. What is a data pipeline?
25. Kafka's role.
26. Spark vs Flink.
27. When do you NOT need Kafka/Spark?
28. ROS 2 for streaming.
29. Schema design basics.
30. Normalization vs denormalization.

31. What is partitioning?
32. Indexing trade-offs.
33. Why partition sensor data by time?
34. Timestamp management pitfalls.
35. Multi-rate synchronization.
36. Resampling vs interpolation.
37. Handling missing data.
38. Why not interpolate GPS gaps?
39. Outlier handling.
40. Data-quality dimensions.
41. Validation at ingestion.
42. Duplicate detection.
43. What is a feature store?
44. Offline vs online features.
45. Train/serve skew.
46. Point-in-time joins.
47. Git vs DVC.
48. What does MLflow track?
49. Reproducibility in research.
50. Encryption at rest vs in transit.

13.2 30 viva questions

1. Design Trinetra-AI's storage tiers.
2. Justify MCAP-for-capture, Parquet-for-compute.
3. Write a TimescaleDB hypertable DDL.
4. Design the sensor time-series schema.
5. When normalize vs denormalize here?
6. Partitioning strategy for readings.
7. Compute storage for a 1-hour multi-sensor session.
8. Design the sync-and-align step.
9. Why is timing error mistaken for sensor error?
10. Data-quality contract for IMU.
11. Detect a clock reset.
12. Ensure train/serve feature parity.
13. Version a dataset with DVC.
14. Track an ablation with MLflow.
15. Batch vs streaming pipeline for this project.
16. Right-size the tooling (avoid over-engineering).
17. Ingest KITTI into the canonical schema.

18. Adapter pattern for many datasets.
19. Handle a dropped-message scenario.
20. Online feature cache design (Redis).
21. Continuous aggregate for a dashboard.
22. Index for "session X, time range" queries.
23. Backup strategy (3-2-1).
24. Secure a trajectory corpus.
25. Throughput budget at 200 Hz.
26. Compression choice & trade-off.
27. Immutable raw layer rationale.
28. Point-in-time correctness.
29. Data lineage/reproducibility.
30. Roadmap from data to visualization.

13.3 20 coding questions

1. Write a synced DataFrame to Parquet (Zstd).
2. Read selected columns from Parquet.
3. Record ROS 2 topics to MCAP.
4. Subscribe to `/imu` and buffer (rclpy).
5. Create a TimescaleDB hypertable + index.
6. Bulk-load Parquet into Postgres.
7. Align multi-rate streams to a grid.
8. Interpolate slow channels; mask gaps.
9. Validate an IMU session (range/rate/monotonic).
10. Detect duplicate (t, sensor) rows.
11. Compute windowed features (shared fn).
12. Cache latest features in Redis.
13. Continuous-aggregate downsample query.
14. Time-range query for a session.
15. Dataset adapter → canonical schema.
16. DVC-track a dataset & push to remote.
17. Log a run to MLflow.
18. Compute a data-quality report.
19. Benchmark raw vs Parquet size.
20. Measure ingest throughput vs sensor rate.

14 Summary, Glossary & Roadmap

14.1 Summary

Data engineering is the reproducible foundation beneath Trinetra-AI's intelligence. A navigation system produces multi-rate, high-volume sensor data that must be captured (ROS 2 → MCAP), stored (Parquet lake + TimescaleDB + Postgres), modelled (normalized metadata, denormalized partitioned readings), cleaned and time-aligned (the make-or-break step), quality-checked (data contracts), served through a feature store with train/serve parity, and versioned (Git + DVC + MLflow) for reproducibility. The guiding principle is *right-sizing*: use the lean ROS 2 → MCAP → pandas → Parquet/TimescaleDB stack for a single-vehicle thesis, and reserve Kafka/Spark/Flink for genuine scale. Done well, this layer makes every downstream result trustworthy and repeatable.

14.2 Glossary

Columnar — column-wise storage (Parquet/ORC) for analytics. **Data contract** — versioned validation rules for a dataset. **DVC** — Git-linked large-data/model versioning. **ELT/ETL** — load-then-transform / transform-then-load. **Feature store** — system serving identical offline/online features. **Hypertable** — TimescaleDB's auto-time-partitioned table. **MCAP** — indexed, compressed ROS 2 log container (default since Iron). **Parquet** — compressed columnar file format. **Partitioning** — splitting a table by key (time/session). **Point-in-time join** — leakage-free feature/label join. **Train/serve skew** — mismatch between training and inference features.

14.3 Roadmap: Data Engineering → Visualization

Hands to	What Data Engineering delivers
Visualization (next)	clean, synced, queryable time-series + features to plot: trajectories, sensor health, error-over-time, dashboards
AI Modules (Phase 9)	versioned, quality-checked feature tables with train/serve parity
Fusion & Navigation (7–8)	real-time synced sensor streams via the online pipeline
The thesis	reproducible datasets & runs (DVC/MLflow) that make every result defensible

Why Visualization comes next. Once the data is captured, cleaned, stored, and served reliably, the natural next step is to *see* it: trajectory plots on a map, sensor-health dashboards, error-over-time curves, and live monitoring. Visualization consumes exactly the queryable time-series and feature tables this phase produced — the TimescaleDB continuous aggregates and Parquet feature store become the data source for every chart and dashboard. Good data engineering is what makes good visualization possible.

End of Phase 11 notes. Next: Visualization — turning the engineered data into trajectories, dashboards, and live monitoring.